

Experiment/ Case Study No.: 7

1. Experiment Title:

2. Software/Tools Required: VS Code, Web Browser, HTML, CSS, JavaScript

3. Experiment Program Code:

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>To-Do List App</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1>To-Do List</h1>
    <div class="input-section">
      <input type="text" id="task-input" placeholder="Add a new task...">
      <button id="add-btn">Add Task</button>
    </div>
    <ul id="task-list">
      <!-- Tasks will be dynamically inserted here -->
    </ul>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

script.js

```
document.addEventListener('DOMContentLoaded', () => {
  const taskInput = document.getElementById('task-input');
  const addBtn = document.getElementById('add-btn');
  const taskList = document.getElementById('task-list');

  // 1. Add Task feature
  addBtn.addEventListener('click', () => {
    const text = taskInput.value.trim();
    if (text !== '') {
      addTask(text);
      taskInput.value = '';
    }
  });

  // 2. Handle Edit and Delete using Event Delegation (DOM Traversal)
  taskList.addEventListener('click', (e) => {
    const target = e.target;

    // DOM Traversal: find the closest list item parent
    const li = target.closest('li');
    if (!li) return;

    // DELETE feature
    if (target.classList.contains('delete-btn')) {
      // DOM Updates: remove the element
      li.remove();
    }

    // EDIT feature
    else if (target.classList.contains('edit-btn')) {
      const isEditing = li.classList.contains('edit-mode');
      const taskTextSpan = li.querySelector('.task-text');
      const editInput = li.querySelector('.edit-input');

      if (isEditing) {
        // Save the changes and update DOM
        const newText = editInput.value.trim();
        if (newText !== '') {
          taskTextSpan.textContent = newText;
        }
        li.classList.remove('edit-mode');
        target.textContent = 'Edit';
      } else {
        // Enter edit mode
        li.classList.add('edit-mode');
        editInput.value = taskTextSpan.textContent;
        editInput.focus();
        target.textContent = 'Save';
      }
    }
  });

  // Helper function to create DOM elements and update the list
```

```

function addTask(text) {
  // Create elements
  const li = document.createElement('li');

  const taskTextSpan = document.createElement('span');
  taskTextSpan.className = 'task-text';
  taskTextSpan.textContent = text;

  const editInput = document.createElement('input');
  editInput.type = 'text';
  editInput.className = 'edit-input';

  const actionsDiv = document.createElement('div');
  actionsDiv.className = 'actions';

  const editBtn = document.createElement('button');
  editBtn.className = 'edit-btn';
  editBtn.textContent = 'Edit';

  const deleteBtn = document.createElement('button');
  deleteBtn.className = 'delete-btn';
  deleteBtn.textContent = 'Delete';

  // DOM Updates: assemble the elements
  actionsDiv.appendChild(editBtn);
  actionsDiv.appendChild(deleteBtn);

  li.appendChild(taskTextSpan);
  li.appendChild(editInput);
  li.appendChild(actionsDiv);

  // Add the task to the list
  taskList.appendChild(li);
}
});

```

style.css

```

body {
  font-family: Arial, sans-serif;
  background-color: #f2f2f2;
  color: #333;
  padding: 20px;
}

.container {
  max-width: 500px;
  margin: 0 auto;
  background: #fff;
  padding: 20px;
  border: 1px solid #ccc;
}

h1 {
  color: #2e7d32; /* Classic green theme heading */
  text-align: center;
}

.input-section {
  display: flex;
  gap: 10px;
  margin-bottom: 20px;
}

input[type="text"] {
  flex-grow: 1;
  padding: 8px;
  border: 1px solid #ccc;
  font-family: Arial, sans-serif;
}

button {
  padding: 8px 15px;
  background-color: #4CAF50; /* Green button */
  color: white;
  border: 1px solid #2e7d32;
  cursor: pointer;
}

button:hover {
  background-color: #45a049;
}

ul {
  list-style-type: none;
}

```

```

padding: 0;
}

li {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 10px;
  border-bottom: 1px dashed #ccc;
  background-color: #f9f9f9;
  margin-bottom: 5px;
}

.task-text {
  flex-grow: 1;
  margin-right: 10px;
}

.actions button {
  padding: 5px 10px;
  margin-left: 5px;
  font-size: 12px;
}

.actions .edit-btn {
  background-color: #fbc02d;
  border-color: #f9a825;
  color: #333;
}

.actions .edit-btn:hover {
  background-color: #f9a825;
}

.actions .delete-btn {
  background-color: #e53935;
  border-color: #c62828;
}

.actions .delete-btn:hover {
  background-color: #c62828;
}

/* Edit mode styles */
.edit-mode .task-text {
  display: none;
}

.edit-input {
  display: none;
  flex-grow: 1;
  margin-right: 10px;
  padding: 5px;
}

.edit-mode .edit-input {
  display: inline-block;
}

.edit-mode .edit-btn {
  background-color: #4CAF50; /* Green to indicate save */
  border-color: #2e7d32;
  color: white;
}

.edit-mode .edit-btn:hover {
  background-color: #45a049;
}

```

4. Output:

alt text

5. Case Study Title:

6. Case Study Program Code:

case_study.html

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Event Handling Case Study</title>
  <style>
    /* Classic Green Theme CSS */

```

```
body {
  font-family: Arial, sans-serif;
  background-color: #f2f2f2;
  color: #333;
  padding: 20px;
  margin: 0;
}

.container {
  max-width: 600px;
  margin: 0 auto;
  background: #fff;
  padding: 20px 30px;
  border: 1px solid #2e7d32;
  border-top: 5px solid #2e7d32;
}

h2 {
  color: #2e7d32;
  text-align: center;
  margin-bottom: 5px;
  border-bottom: 2px solid #ccc;
  padding-bottom: 10px;
}

.mandatory-msg {
  color: #d32f2f;
  font-size: 14px;
  text-align: right;
  margin-top: 0;
}

#status-panel {
  background-color: #e8f5e9;
  border: 1px solid #4caf50;
  padding: 10px;
  margin-bottom: 20px;
  font-weight: bold;
  color: #2e7d32;
}

#event-message {
  color: #d32f2f;
  font-style: italic;
}

.form-group {
  margin-bottom: 15px;
}

label {
  display: block;
  font-weight: bold;
  margin-bottom: 5px;
}

label .req {
  color: #d32f2f;
}

input[type="text"],
input[type="email"],
select,
textarea {
  width: 100%;
  padding: 8px;
  border: 1px solid #ccc;
  box-sizing: border-box;
  font-family: Arial, sans-serif;
}

textarea {
  resize: vertical;
  height: 80px;
}

.dob-group {
  display: flex;
  gap: 10px;
}

.dob-group select {
  flex: 1;
}

input[type="radio"] {
```

```

        margin-right: 5px;
    }

    label.inline {
        display: inline-block;
        font-weight: normal;
        margin-right: 15px;
    }

    .error-msg {
        color: #d32f2f;
        font-size: 12px;
        min-height: 14px;
        margin-top: 3px;
    }

    /* Dynamic classes added by JavaScript */
    input.focused, select.focused, textarea.focused {
        outline: none;
        border-color: #4CAF50;
        background-color: #f1f8e9;
    }

    input.error, select.error {
        border-color: #d32f2f;
        background-color: #ffebee;
    }

    .form-actions {
        text-align: center;
        margin-top: 20px;
    }

    button {
        padding: 10px 20px;
        border: none;
        color: white;
        font-size: 16px;
        cursor: pointer;
        margin: 0 5px;
    }

    #submit-btn {
        background-color: #4CAF50;
    }

    #submit-btn:hover {
        background-color: #45a049;
    }

    #reset-btn {
        background-color: #757575;
    }

    #reset-btn:hover {
        background-color: #616161;
    }

    #output-panel {
        margin-top: 30px;
        padding: 15px;
        background-color: #f9f9f9;
        border: 1px solid #ccc;
        border-left: 5px solid #2e7d32;
    }

    #output-panel h3 {
        margin-top: 0;
        color: #2e7d32;
    }

    .hidden {
        display: none;
    }
</style>
</head>
<body>

<div class="container">
    <h2>Student Registration Form</h2>
    <p class="mandatory-msg">* Mandatory to fill</p>

    <!-- Live Form Status Panel -->
    <div id="status-panel">
        Form Status: <span id="event-message">Ready for input...</span>
    </div>

```

```

<form id="reg-form">
  <div class="form-group">
    <label for="fname">First Name <span class="req">*</span></label>
    <input type="text" id="fname" name="fname">
    <div class="error-msg" id="fname-err"></div>
  </div>

  <div class="form-group">
    <label for="lname">Last Name <span class="req">*</span></label>
    <input type="text" id="lname" name="lname">
    <div class="error-msg" id="lname-err"></div>
  </div>

  <div class="form-group">
    <label>Birthday <span class="req">*</span></label>
    <div class="dob-group">
      <select id="day" name="day"><option value="">Day</option></select>
      <select id="month" name="month"><option value="">Month</option></select>
      <select id="year" name="year"><option value="">Year</option></select>
    </div>
    <div class="error-msg" id="dob-err"></div>
  </div>

  <div class="form-group">
    <label for="email">Email <span class="req">*</span></label>
    <input type="email" id="email" name="email">
    <div class="error-msg" id="email-err"></div>
  </div>

  <div class="form-group">
    <label for="mobile">Mobile Number <span class="req">*</span></label>
    <input type="text" id="mobile" name="mobile">
    <div class="error-msg" id="mobile-err"></div>
  </div>

  <div class="form-group">
    <label>Gender <span class="req">*</span></label>
    <input type="radio" id="male" name="gender" value="Male"> <label for="male" class="inline">Male</label>
    <input type="radio" id="female" name="gender" value="Female"> <label for="female" class="inline">Female</label>
    <input type="radio" id="other" name="gender" value="Other"> <label for="other" class="inline">Other</label>
    <div class="error-msg" id="gender-err"></div>
  </div>

  <div class="form-group">
    <label for="course">Course <span class="req">*</span></label>
    <select id="course" name="course">
      <option value="">--Select Course--</option>
      <option value="BCA">BCA</option>
      <option value="MCA">MCA</option>
      <option value="BTech">B.Tech</option>
    </select>
    <div class="error-msg" id="course-err"></div>
  </div>

  <div class="form-group">
    <label for="address">Address</label>
    <textarea id="address" name="address"></textarea>
  </div>

  <div class="form-actions">
    <button type="submit" id="submit-btn">Submit</button>
    <button type="reset" id="reset-btn">Reset</button>
  </div>
</form>

<!-- Area to display submitted information -->
<div id="output-panel" class="hidden">
  <h3>Submitted Student Details</h3>
  <div id="output-content"></div>
</div>
</div>

<script>
  document.addEventListener('DOMContentLoaded', () => {
    // DOM MANIPULATION: Access form elements using document.getElementById and querySelectorAll
    const form = document.getElementById('reg-form');
    const eventMessage = document.getElementById('event-message');
    const outputPanel = document.getElementById('output-panel');
    const outputContent = document.getElementById('output-content');
    const resetBtn = document.getElementById('reset-btn');
    const submitBtn = document.getElementById('submit-btn');

    const textInputs = form.querySelectorAll('input[type="text"], input[type="email"], textarea');
    const selects = form.querySelectorAll('select');
    const radios = form.querySelectorAll('input[type="radio"]');
  });

```

```

// DOM MANIPULATION: Populate Birthday dropdowns dynamically (document.createElement, appendChild)
const daySelect = document.getElementById('day');
for (let i = 1; i <= 31; i++) {
  let option = document.createElement('option');
  option.value = i; option.textContent = i;
  daySelect.appendChild(option);
}

const monthSelect = document.getElementById('month');
const months = ['Jan', 'Feb', 'Mar', 'Apr', 'May', 'Jun', 'Jul', 'Aug', 'Sep', 'Oct', 'Nov', 'Dec'];
months.forEach((m, idx) => {
  let option = document.createElement('option');
  option.value = idx + 1; option.textContent = m;
  monthSelect.appendChild(option);
});

const yearSelect = document.getElementById('year');
for (let i = 2024; i >= 1990; i--) {
  let option = document.createElement('option');
  option.value = i; option.textContent = i;
  yearSelect.appendChild(option);
}

// Helper function to update the event status message dynamically
const updateEventMessage = (msg) => {
  eventMessage.textContent = msg; // DOM manipulation: Change textContent
};

// Event Handling logic below

textInputs.forEach(input => {
  // 1. FOCUS EVENT
  input.addEventListener('focus', (e) => {
    // DOM Manipulation: element.classList.add()
    e.target.classList.add('focused');
    updateEventMessage(`FOCUS Event: You are now focused on ${e.target.id} field.`);
  });

  // 2. BLUR EVENT
  input.addEventListener('blur', (e) => {
    // DOM Manipulation: element.classList.remove()
    e.target.classList.remove('focused');
    updateEventMessage(`BLUR Event: You left the ${e.target.id} field.`);
    validateTextField(e.target);
  });

  // 3. INPUT EVENT
  input.addEventListener('input', (e) => {
    updateEventMessage(`INPUT Event: Typing in ${e.target.id} - Current length: ${e.target.value.length}`);
  });
});

selects.forEach(select => {
  // 4. CHANGE EVENT
  select.addEventListener('change', (e) => {
    updateEventMessage(`CHANGE Event: Selected value '${e.target.value}' for ${e.target.id}`);
    // Clear dropdown errors if valid
    if (['day', 'month', 'year'].includes(e.target.id)) {
      document.getElementById('dob-err').textContent = '';
    } else if (e.target.id === 'course') {
      document.getElementById('course-err').textContent = '';
    }
  });
});

radios.forEach(radio => {
  // 4. CHANGE EVENT (for radios)
  radio.addEventListener('change', (e) => {
    updateEventMessage(`CHANGE Event: Selected Gender - ${e.target.value}`);
    document.getElementById('gender-err').textContent = ''; // Clear error on change
  });
});

// 5. CLICK EVENT (Demonstration on Submit and Reset buttons)
submitBtn.addEventListener('click', () => {
  // Note: the submit event on the form actually fires after this click event
  updateEventMessage(`CLICK Event: Submit button was clicked.`);
});

resetBtn.addEventListener('click', () => {
  updateEventMessage(`CLICK Event: Reset button was clicked.`);
});

// 6. RESET EVENT
form.addEventListener('reset', () => {
  updateEventMessage(`RESET Event: Form has been cleared.`);
});

```

```

// DOM Manipulation: clear validation messages and errors
document.querySelectorAll('.error-msg').forEach(el => el.textContent = '');
document.querySelectorAll('.error').forEach(el => el.classList.remove('error'));

outputPanel.classList.add('hidden'); // hide output panel
outputContent.innerHTML = '';
});

// 7. SUBMIT EVENT
form.addEventListener('submit', (e) => {
  e.preventDefault(); // Prevent default browser submission

  let isValid = true;

  // Validate all text fields
  textInputs.forEach(input => {
    if (input.id !== 'address' && !validateTextField(input)) {
      isValid = false;
    }
  });

  // Validate Birthday
  if (!daySelect.value || !monthSelect.value || !yearSelect.value) {
    document.getElementById('dob-err').textContent = 'Please select a complete birthday.';
    isValid = false;
  }

  // Validate Course
  if (!document.getElementById('course').value) {
    document.getElementById('course-err').textContent = 'Please select a course.';
    isValid = false;
  }

  // Validate Gender
  const genderSelected = document.querySelector('input[name="gender"]:checked');
  if (!genderSelected) {
    document.getElementById('gender-err').textContent = 'Please select a gender.';
    isValid = false;
  }

  if (isValid) {
    updateEventMessage('SUBMIT Event: Form validation passed. Displaying data.');
```

// DOM MANIPULATION: element.classList.remove() to show output

```
outputPanel.classList.remove('hidden');
```

// DOM MANIPULATION: element.innerHTML to reset content

```
outputContent.innerHTML = '';
```

```

    const details = [
      `Full Name: ${document.getElementById('fname').value} ${document.getElementById('lname').value}`,
      `Birthday: ${daySelect.value}/${monthSelect.value}/${yearSelect.value}`,
      `Email: ${document.getElementById('email').value}`,
      `Mobile Number: ${document.getElementById('mobile').value}`,
      `Gender: ${genderSelected.value}`,
      `Course: ${document.getElementById('course').value}`,
      `Address: ${document.getElementById('address').value}`
    ];

    // DOM MANIPULATION: document.createElement() and appendChild() to build details list
    const ul = document.createElement('ul');
    details.forEach(text => {
      const li = document.createElement('li');
      li.textContent = text;
      ul.appendChild(li);
    });
    outputContent.appendChild(ul);

  } else {
    updateEventMessage('SUBMIT Event: Validation failed. Fix highlighted errors.');
```

outputPanel.classList.add('hidden');

```

  }
});

// Helper function for individual text field validation
function validateTextField(element) {
  let errorMsg = '';
  let isValid = true;
  const val = element.value.trim(); // DOM Manipulation: element.value

  if (element.id === 'fname' || element.id === 'lname') {
    if (val === '') {
      errorMsg = 'This field cannot be empty.';
      isValid = false;
    }
  } else if (element.id === 'email') {

```

```

        const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
        if (val === '' || !emailPattern.test(val)) {
            errorMsg = 'Please enter a valid email address.';
            isValid = false;
        }
    } else if (element.id === 'mobile') {
        const mobilePattern = /^\d{10}$/;
        if (!mobilePattern.test(val)) {
            errorMsg = 'Mobile number must be exactly 10 digits.';
            isValid = false;
        }
    }

    // Find the corresponding error div and update it
    const errDiv = document.getElementById(`${element.id}-err`);
    if (errDiv) {
        errDiv.textContent = errorMsg; // DOM manipulation
    }

    // Apply error class
    if (!isValid) {
        element.classList.add('error');
    } else {
        element.classList.remove('error');
    }

    return isValid;
}
});
</script>

</body>
</html>

```

7. Output:

alt text

8. Result/Conclusion:

The experiment successfully demonstrated the use of JavaScript for DOM manipulation and event delegation by building a functional To-Do List application. Key features such as adding, editing, and deleting tasks were implemented dynamically without reloading the page. The case study extended event handling concepts by building a comprehensive student registration form that tracks focus, blur, input, change, submit, and reset events with live validation. The application highlights efficient ways to interact with the DOM and handle complex user interactions.